今日及接下来几次课程的内容安排

- 单周期微体系结构（Single-cycle Microarchitectures）
- 多周期与微程序化微体系结构（Multi-cycle and Microprogrammed Microarchitectures）
- 流水线技术（Pipelining）
- 流水线中的问题：控制与数据相关性处理、状态维护与恢复等（Issues in Pipelining: Control & Data Dependence Handling, State Maintenance and Recovery, …）

回顾

- **微程序化总结：**
  - 水平微代码 vs. 垂直微代码
  - 纳米代码 vs. 微代码
- **流水线：**
  - 理想流水线的基本思想与特性
  - 流水线数据路径与控制
  - 流水线设计中的问题
  - 资源争用
  - 依赖关系及其类型
    - 控制依赖 vs. 数据依赖（流依赖、反依赖、输出依赖）
  - 处理数据依赖的五种基本方法
  - 依赖检测
    - 互锁机制
    - 记分板 vs. 组合逻辑

回顾：流水线设计中的问题

- 平衡流水线各阶段的工作
  - 有多少个阶段，每个阶段做什么？
- 保持流水线的正确性、流动性和完整性，尤其在出现干扰流水线流动的事件时（Keeping the pipeline correct, moving, and full in the presence of events that disrupt pipeline flow）
  - 处理依赖关系（Handling dependencies）
    - 数据依赖（Data）
    - 控制依赖（Control）
  - 处理资源争用（Handling resource contention）
  - 处理长延迟（多周期）操作（Handling long-latency (multi-cycle) operations）
- 处理异常和中断（Handling exceptions, interrupts）
- 进阶：提高流水线吞吐量
  - 最小化流水线停顿（Minimizing stalls）

回顾：依赖关系及其类型

- 也叫做“依赖性”或 “冒险”（hazard）。
- 依赖关系决定了指令之间的执行顺序要求。
- 有两种类型：
  - 数据依赖（Data dependence）
  - 控制依赖（Control dependence）
- 资源争用有时被称为资源依赖（Resource dependence）。
  - 然而，这并不是程序语义上根本的要求，因此我们将单独处理它。

回顾：互锁（Interlocking）

- 流水线处理器中指令之间依赖的检测，以保证正确执行。
- 基于软件的互锁 与 基于硬件的互锁。
- MIPS的缩写是什么？
  - “无内部互锁流水级的微处理器”(Microprocessor without interlocked piped stages)

回顾：一旦在硬件中检测到依赖，怎么办？

- 选择：
  - 立即停顿依赖指令。
  - 仅在必要时停顿依赖指令——数据转发/绕过。
  - 选择其他方式…… （乱序执行）

数据前送/旁路(Data Forwarding/Bypassing)

- 问题：消费者（依赖）指令必须在解码阶段等待，直到生产者指令将其值写入寄存器文件。

数据前送/旁路(Data Forwarding/Bypassing)

- 问题：消费者（依赖）指令必须在解码阶段等待，直到生产者指令将其值写入寄存器文件。
- 目标：我们不希望不必要地停顿流水线。
- 观察：消费者指令所需的数据值可以直接从流水线后续阶段提供（而不仅仅是从寄存器文件中提供）。

数据前送/旁路(Data Forwarding/Bypassing)

- 问题：消费者（依赖）指令必须在解码阶段等待，直到生产者指令将其值写入寄存器文件。
- 目标：我们不希望不必要地停顿流水线。
- 观察：消费者指令所需的数据值可以直接从流水线后续阶段提供（而不仅仅是从寄存器文件中提供）。
- 想法：添加额外的依赖检测逻辑和数据前递路径（总线），在生产者值可用后，立即将其提供给消费者。
- 好处：消费者可以继续在流水线中移动，直到值可供使用，从而减少停顿。

数据依赖的特殊情况

- **控制依赖**
  - 程序计数器（Instruction Pointer / Program Counter）上的数据依赖。

控制依赖 （Control Dependence）

- **问题：下一周期的取指令程序计数器（Fetch PC）应该是什么？**
  - **回答**：下一条指令的地址
  - 所有指令都控制依赖于之前的指令。为什么？
- **如果获取的指令是非控制流指令：**
  - 下一条取指令的程序计数器（Fetch PC）是下一条顺序指令的地址。
  - 如果知道取指令的大小，就容易确定。
- **如果获取的指令是控制流指令：**
  - 如何确定下一条取指令的程序计数器（Fetch PC）？
- **如何知道获取的指令是否是控制流指令？**

今日的内容

- 数据依赖处理：更深入的探讨与实现

记住：数据依赖类型

Flow dependence

$r_3 \leftarrow r_1$ op r_2

$r_5 \leftarrow r_3$ op r_4

Read-after-Write (RAW)

Anti dependence

$r_3 \leftarrow r_1$ op r_2

$r_1 \leftarrow r_4$ op r_5

Write-after-Read (WAR)

Output-dependence

$r_3 \leftarrow r_1$ op r_2

$r_5 \leftarrow r_3$ op r_4

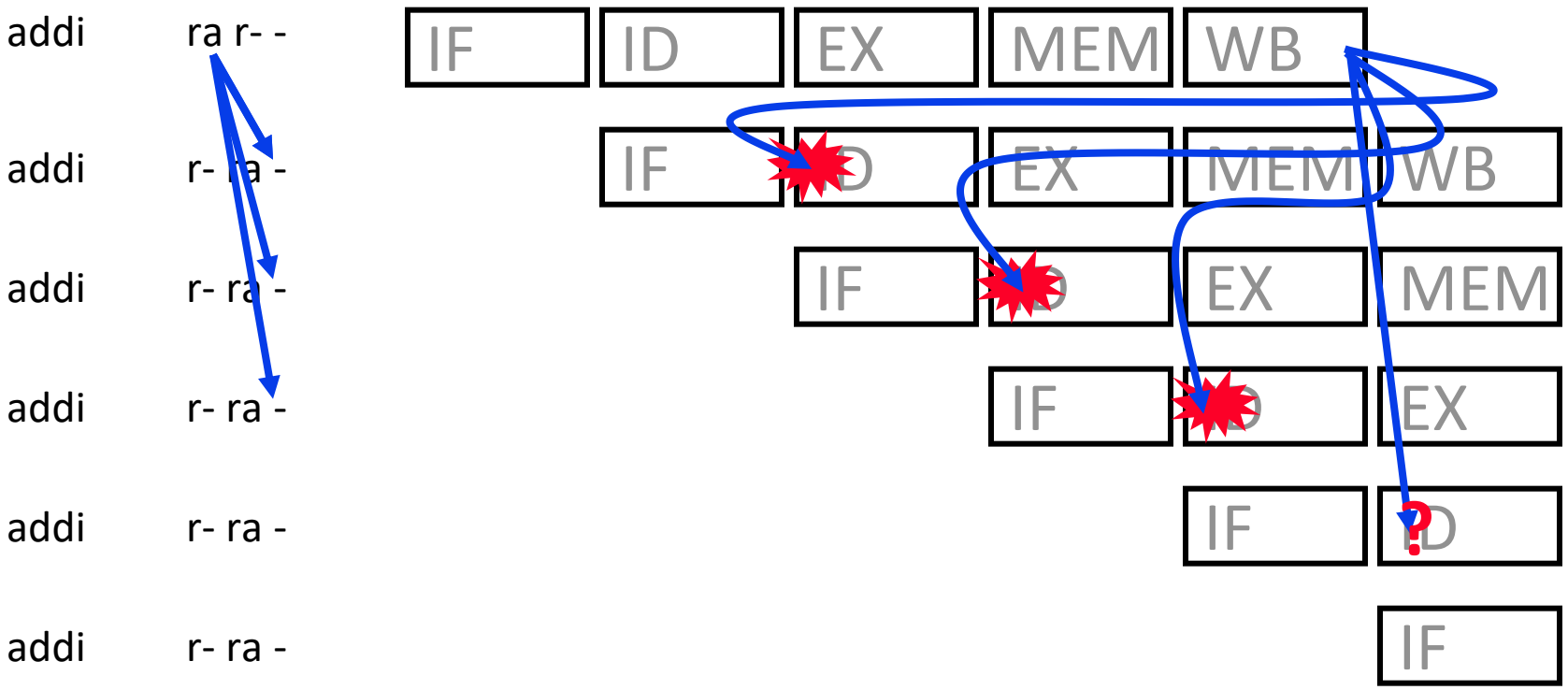
$r_3 \leftarrow r_6$ op r_7

Write-after-Write (WAW)

记住：如何处理数据依赖

- **反依赖和输出依赖 比较容易处理：**
  - 在一个阶段写入目标并按程序顺序执行。
- **流依赖更复杂**
- **处理流依赖的五种基本方法：**
  - **检测并等待**，直到值在寄存器文件中可用。
  - **检测并转发/绕过** 数据到依赖的指令。
  - **在软件层面消除依赖**：无需硬件检测依赖。
  - **预测所需的值**，进行“猜测性”执行，并验证。
  - **做其他事情**（细粒度的多线程）：不需要检测。

RAW 依赖处理

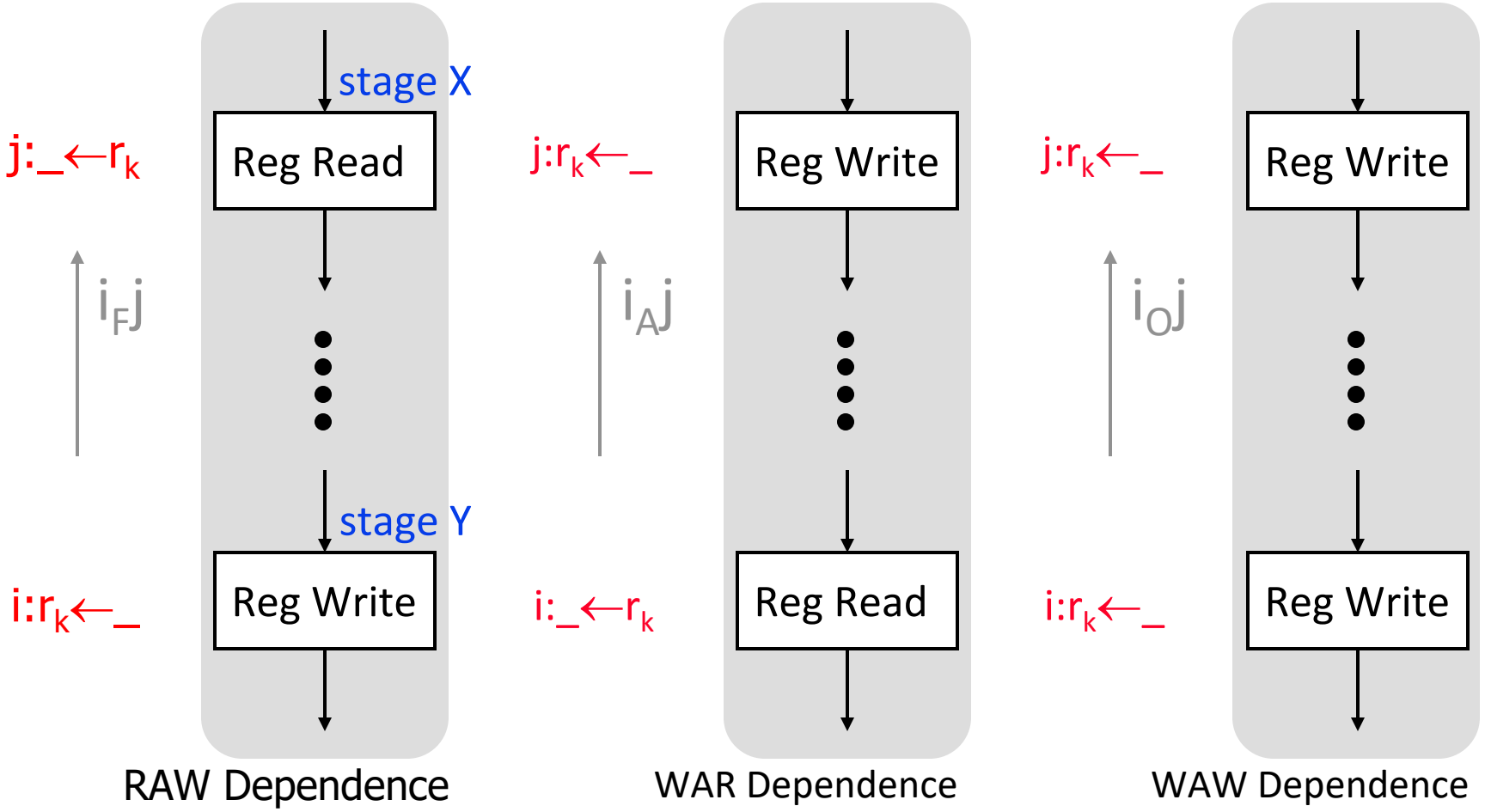

addi ra r- -
addi r- ra -
addi r- ra -
addi r- ra -
addi r- ra -
addi r- ra -
IF
ID
EX
MEM
WB
?

寄存器数据依赖分析

	R/I-Type	LW	SW	Br	J	Jr
IF						
ID	read RF	read RF	read RF	read RF		read RF
EX						
MEM						
WB	write RF	write RF				

- 对于给定的流水线，何时会出现两个数据依赖指令之间的潜在冲突？
  - 依赖类型：RAW、WAR、WAW?
  - 涉及的指令类型?
  - 两条指令之间的距离?

流水线的安全与不安全移动



dist(i,j) ≤ dist(X,Y) ⇒ j移动不安全

dist(i,j) > dist(X,Y) ⇒ 安全

流水线的安全与不安全移动

符号	全称
iFj	*i feeds j* (i 给 j 提供数据)
iAj	*i anti-dependent on j*
iOj	*i output-dependent on j*

指令 i：较早进入流水线的指令；指令 j：较晚进入流水线、可能依赖于指令 i 的指令。

流水线的安全与不安全移动（Safe and Unsafe Movement of Pipeline）

这里的“移动”（moving）其实指的是——流水线中指令 j 继续向下推进到下一个阶段（stage）的动作。

流水线的不安全移动-例子

- **场景设定**
  - 流水线阶段：Reg Read 在 ID，Reg Write 在 WB
    ⇒ 结构距离 dist(X,Y)=3（从读到写相隔 3 个节拍）。
  - 两条指令：
    i：ADD r1，r2，r3 （写 r1）
    j：SUB r4，r1，r5 （读 r1）
    因为 j 读的是 i 要写的 r1，所以是 RAW 相关。
  - 发射间距：j 紧跟 i 进入流水（中间没有别的指令）
    ⇒ dist(i,j)=1。

- **统一判定公式**
  - 若 dist(i,j) ⩽ dist(X,Y) ⇒ 不安全（必须想办法拉开距离）
  - 若 dist(i,j) > dist(X,Y) ⇒ 安全
  - 直觉：j 离 i 太近了，就会在 i 写回之前“过早地去读”，读到旧值。

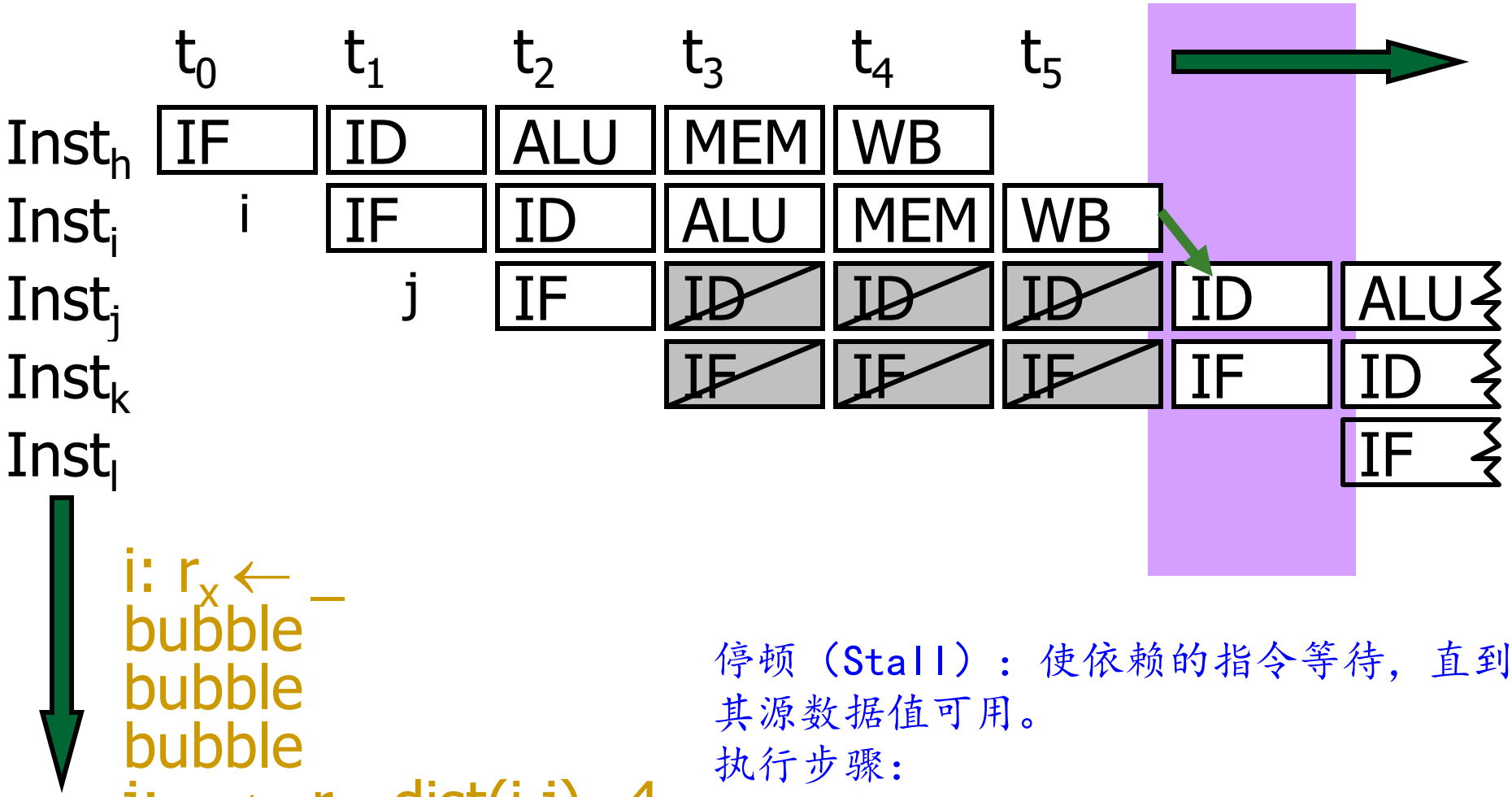
RAW 依赖分析示例

	R/I-Type	LW	SW	Br	J	Jr
IF						
ID	read RF	read RF	read RF	read RF		read RF
EX						
MEM						
WB	write RF	write RF				

- 指令 I_a 和 I_β（其中 I_a 在 I_β 之前）具有 RAW 依赖，当且仅当：
  - I_β（R/I、LW、SW、Br 或 JR）读取了 I_a（R/I 或 LW）写入的寄存器
  - $dist(I_a, I_\beta) \leqslant dist(ID, WB) = 3$

- 思考：那 WAW 和 WAR 依赖呢？

流水线停顿：解决数据依赖



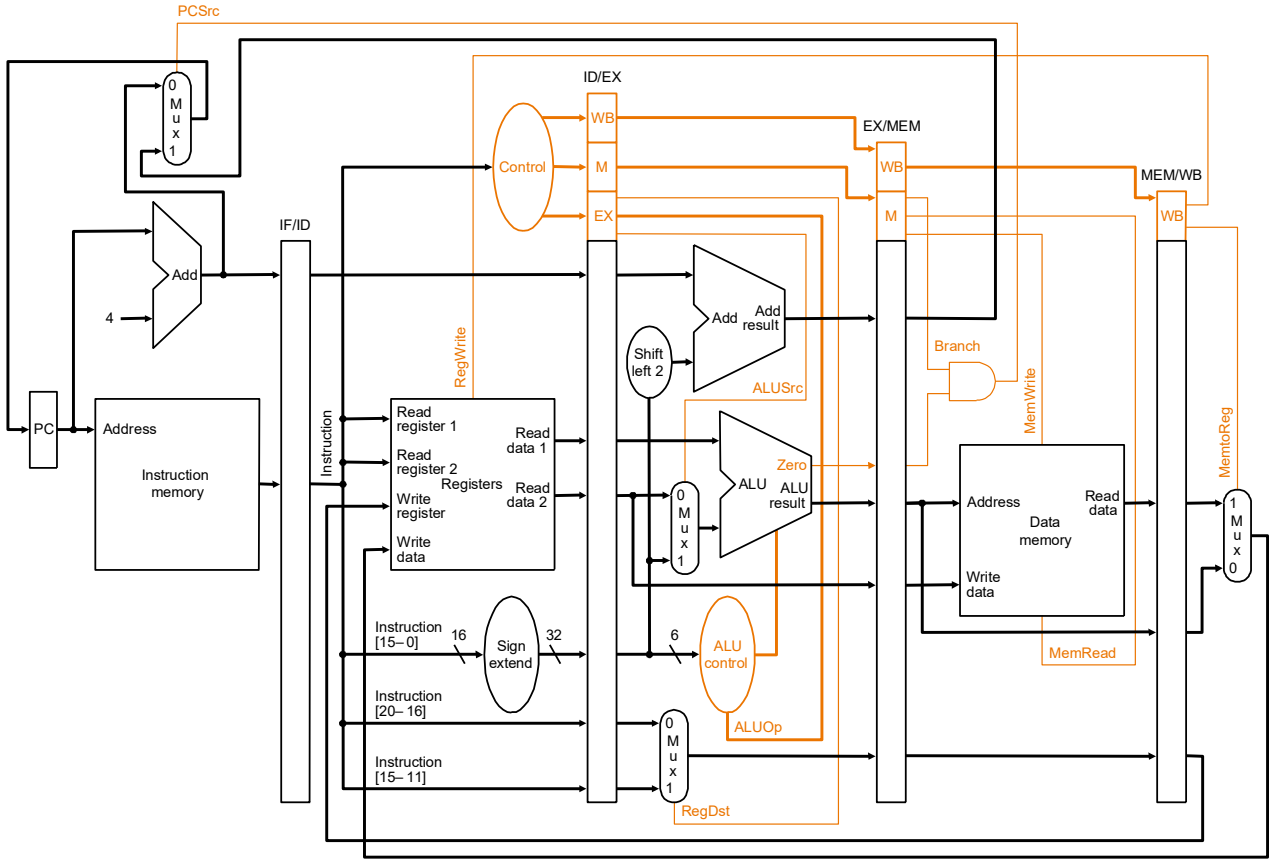
停顿（Stall）：使依赖的指令等待，直到其源数据值可用。

执行步骤：

停止所有上游阶段（up-stream stages）

排空所有下游阶段（down-stream stages）

如何实现流水线停顿（Stalling）



- **停顿（Stall）机制：**
  - 禁用 PC 和 IR 锁存（latching）；确保发生停顿的指令停留在其当前阶段（stage）中，不会前进。
  - 插入 “无效” 指令 或 空操作（NOP）；将这些指令插入到紧随停顿指令之后的阶段中，这些空操作被称为 “气泡（bubbles）”。

停顿条件（Stall Conditions）

- 指令 I_a 和 I_β（其中 I_a 在 I_β 之前）具有 RAW（读后写）依赖 当且仅当：
  - I_β（R/I、LW、SW、Br 或 JR）读取了 I_a（R/I 或 LW）写入的寄存器
  - $dist(I_a, I_\beta) \leqslant dist(ID, WB) = 3$

- 当 I_β 在 ID 阶段 想要读取一个寄存器，而该寄存器将由 I_a 在 EX、MEM 或 WB 阶段 写入时，必须 让 ID 阶段停顿（stall）。

停顿（Stall）对性能的影响

- 每一个停顿周期（stall cycle）都对应于 一个丢失的周期（one lost cycle），
  在这个周期中，没有任何指令可以完成。

- 对于一个包含 N 条指令 且有 S 个停顿周期 的程序：
  平均 CPI（每指令周期数） = (N + S) / N

- S 的大小取决于：
  - RAW（读后写）依赖的频率
  - 依赖指令之间的具体距离
  - 依赖之间的分布
    - 比如若 i_1、i_2 和 i_3 都依赖于 i_0，一旦 i_1 的依赖被解决，则 i_2 和 i_3 通常也能顺利执行。